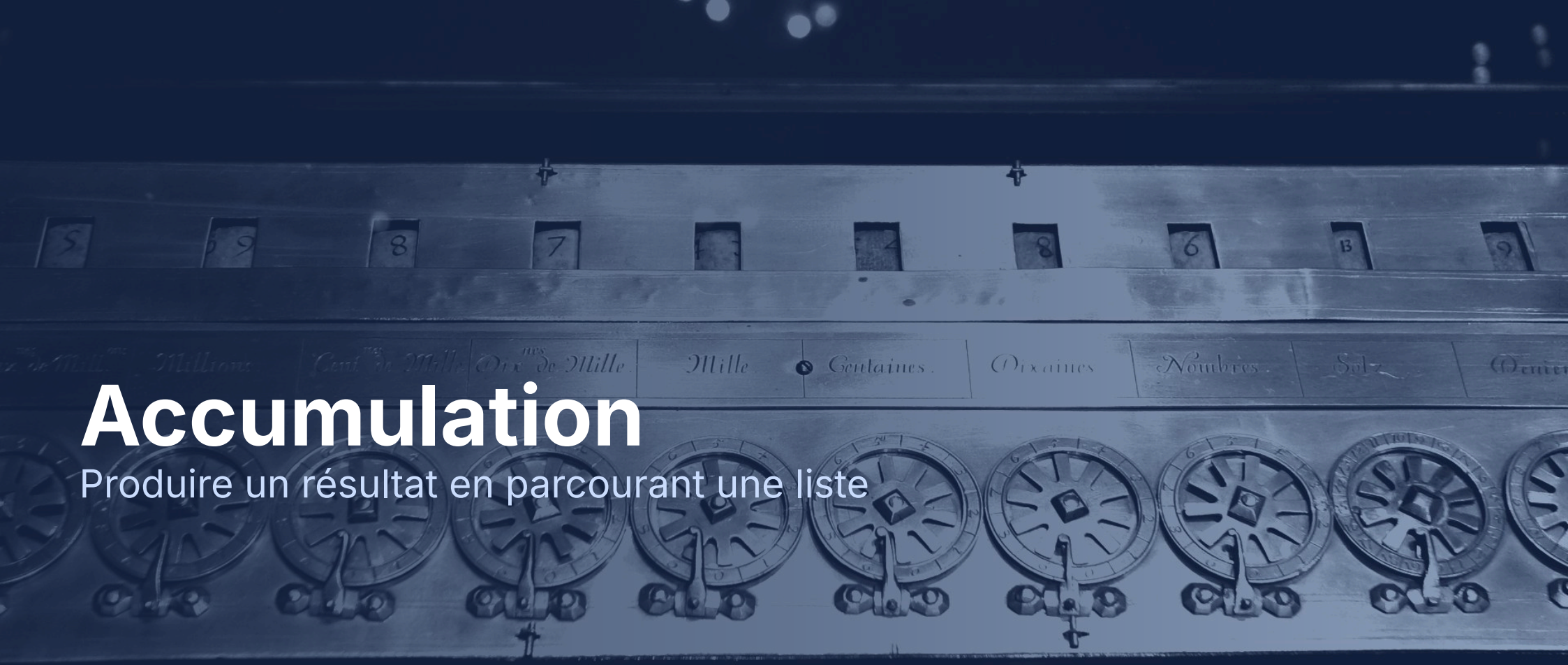


Accumulation

Produire un résultat en parcourant une liste

Nicholas Wolff · CC BY-SA 4.0

Sommaire

1. **Utilité d’un algorithme, d’un programme**
   On se concentre sur le fait qu’un algorithme produit un résultat
2. **Programmes simples**
   Des programmes qui prennent des entrées et produisent un résultat immédiatement
3. **Accumulation**
   Produire un résultat petit à petit
4. **Valeur initiale et opération**
   Pour une opération répétée, on peut toujours trouver une valeur initiale qui correspond
5. **Exercices d’accumulation**
   Six exercices

Utilité d’un algorithme

Un algorithme est une suite finie et non ambiguë d’instructions et d’opérations permettant de résoudre une *classe* de problèmes.

Un algorithme prend une ou des valeurs d’entrée et **produit une ou plusieurs valeurs de sortie** comme solution d’un problème bien défini.

Un algorithme peut être exprimé dans un *langage naturel* (par exemple le français), un *langage de programmation* (par exemple Python), un *algorigramme* ou un circuit électronique.

Utilité d’un programme informatique

Quand un algorithme est exprimé dans un langage de programmation, c’est un programme.

Un programme est une suite finie et non ambiguë d’instructions et d’opérations permettant de résoudre une *classe* de problèmes.

Un programme prend une ou des valeurs d’entrée et **produit une ou plusieurs valeurs de sortie** comme solution d’un problème bien défini.

Sommaire

1 **Utilité d'un algorithme, d'un programme**
On se concentre sur le fait qu'un algorithme produit un résultat

2 **Programmes simples**
Des programmes qui prennent des entrées et produisent un résultat immédiatement

3 **Accumulation**
Produire un résultat petit à petit

4 **Valeur initiale et opération**
Pour une opération répétée, on peut toujours trouver une valeur initiale qui correspond

5 **Exercices d'accumulation**
Six exercices

Additionner deux nombres

On veut *produire un résultat* :

- On isole les entrées au début et la sortie à la fin du programme.
- Le cœur du programme est surligné en jaune. C'est là qu'on produit le résultat à partir des entrées.

```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
```

Additionner deux nombres

On veut *produire un résultat* :

- On isole les entrées au début et la sortie à la fin du programme.
- Le cœur du programme est surligné en jaune. C'est là qu'on produit le résultat à partir des entrées.

```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
3
4 résultat = a + b
```

Additionner deux nombres

On veut *produire un résultat* :

- On isole les entrées au début et la sortie à la fin du programme.
- Le cœur du programme est surligné en jaune. C'est là qu'on produit le résultat à partir des entrées.

```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
3
4 résultat = a + b
5
6 print("Résultat :", résultat)
```

Maximum de deux nombres

On veut produire un résultat :

- On isole les entrées au début et la sortie à la fin de l'algorithme / du programme.
- On introduit une variable qui va contenir le résultat de l'algorithme / du programme.

```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
```

Maximum de deux nombres

On veut produire un résultat :

- On isole les entrées au début et la sortie à la fin de l'algorithme / du programme.
- On introduit une variable qui va contenir le résultat de l'algorithme / du programme.

```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
3
4 if a > b:
5     résultat = a
6 else:
7     résultat = b
```

Maximum de deux nombres

On veut produire un résultat :

- On isole les entrées au début et la sortie à la fin de l'algorithme / du programme.
- On introduit une variable qui va contenir le résultat de l'algorithme / du programme.

```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
3
4 if a > b:
5     résultat = a
6 else:
7     résultat = b
8
9 print("Résultat :", résultat)
```

Maximum de trois nombres


```
1 a = int(input("entrez un nombre : "))
2 b = int(input("entrez un nombre : "))
3 c = int(input("entrez un nombre : "))
```

Maximum de trois nombres


```
1  a = int(input("entrez un nombre : "))
2  b = int(input("entrez un nombre : "))
3  c = int(input("entrez un nombre : "))
4
5  if a > b:
6      résultat = a
7  else:
8      résultat = b
```

Maximum de trois nombres


```
a = int(input("entrez un nombre : "))
b = int(input("entrez un nombre : "))
c = int(input("entrez un nombre : "))

if a > b:
    résultat = a
else:
    résultat = b
if c > résultat:
    résultat = c
```

Maximum de trois nombres


```python
a = int(input("entrez un nombre : "))
b = int(input("entrez un nombre : "))
c = int(input("entrez un nombre : "))

if a > b:
    résultat = a
else:
    résultat = b
if c > résultat:
    résultat = c

print("Résultat :", résultat)
```

Sommaire

1 **Utilité d’un algorithme, d’un programme**
On se concentre sur le fait qu’un algorithme produit un résultat

2 **Programmes simples**
Des programmes qui prennent des entrées et produisent un résultat immédiatement

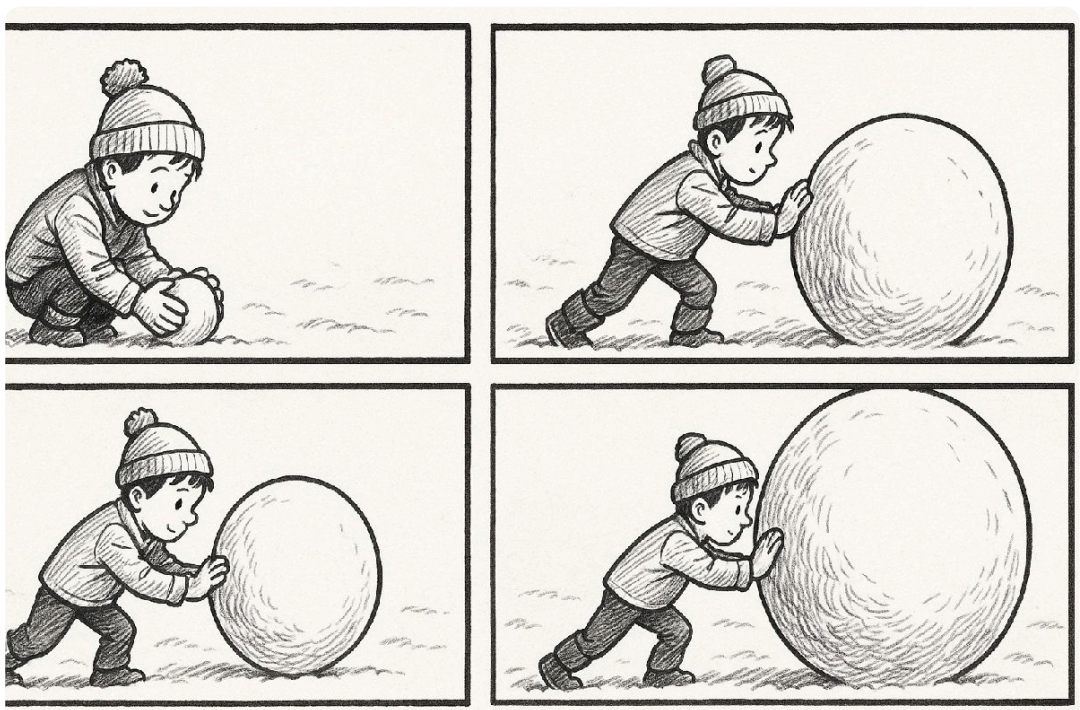
3 **Accumulation**
Produire un résultat petit à petit

4 **Valeur initiale et opération**
Pour une opération répétée, on peut toujours trouver une valeur initiale qui correspond

5 **Exercices d’accumulation**
Six exercices

Accumulation

Technique basée sur l'*itération* permettant de produire un ou plusieurs résultats **petit à petit**.



Que fait ce programme ?


```
s = 0
n = int(input("Entrez un nombre (0 pour terminer) "))
while n != 0:
    s = s + n
    n = int(input("Entrez un nombre (0 pour terminer) "))
print("Résultat :", s)
```

Activité · 6 min

1) Écrire la trace pour le cas où la personne qui utilise le programme répond dans l'ordre : 6, 11, 0.

2) En quelques mots, que fait cet algorithme ? Si vous ne trouvez pas tout de suite, inventer un autre exemple et écrire sa trace.

Que fait ce programme ?

Ce programme produit un résultat en considérant les nombres donnés par la personne qui l'utilise **au fur et à mesure**.
Il **accumule** le résultat dans une variable `s`.

```
1 s = 0
2 n = int(input("Entrez un nombre (0 pour terminer) "))
3 while n != 0:
4     s = s + n
5     n = int(input("Entrez un nombre (0 pour terminer) "))
6 print("Résultat :", s)
```

Activité

1) Écrire la trace pour le cas où la personne qui utilise le programme répond dans l'ordre : 6, 11, 0.

2) En quelques mots, que fait cet algorithme ? Si vous ne trouvez pas tout de suite, inventer un autre exemple et écrire sa trace.

Que fait ce programme (suite) ?


```
s = 0
n = int(input("Entrez un nombre (0 pour terminer) "))
while n != 0:          # n différent de 0
    s = s + n
    n = int(input("Entrez un nombre (0 pour terminer) "))
print("Résultat :", s)
```

Pour calculer la somme par accumulation :

- On introduit une *variable d'accumulation* ; ici, on a décidé de la nommer `s` pour « somme ».
- On l'initialise. Ici à `0`, parce que 0, c'est la somme de rien.
- À chaque itération, on ajoute le nombre n qu'on rencontre à la variable `s`.

On aura toujours la même forme quand on voudra produire un résultat par accumulation.

Sommaire

1. **Utilité d'un algorithme, d'un programme**
   On se concentre sur le fait qu'un algorithme produit un résultat
2. **Programmes simples**
   Des programmes qui prennent des entrées et produisent un résultat immédiatement
3. **Accumulation**
   Produire un résultat petit à petit
4. **Valeur initiale et opération**
   Pour une opération répétée, on peut toujours trouver une valeur initiale qui correspond
5. **Exercices d'accumulation**
   Six exercices

Accumulation : valeur initiale et opération

Activité · 5 min

On cherche la valeur initiale et l'opération qui permettent de calculer un résultat par accumulation pour différents algorithmes.

Algorithme	Valeur initiale	Opération (répétée)	Code de la valeur initiale	Code de l'opération (répétée)
Compter les éléments				
Calculer la somme des éléments				
Calculer le produit des éléments				
Fabriquer une liste avec les éléments				
Fabriquer une chaîne de caractères				

Accumulation : valeur initiale et opération

Activité

On cherche la valeur initiale et l'opération qui permettent de calculer un résultat par accumulation pour différents algorithmes.

Algorithme	Valeur initiale	Opération (répétée)	Code de la valeur initiale	Code de l'opération (répétée)
Compter les éléments	0	ajouter un au résultat	`résultat = 0`	`résultat = résultat + 1`
Calculer la somme des éléments	0	ajouter l'élément au résultat	`résultat = 0`	`résultat = résultat + élément`
Calculer le produit des éléments	1	multiplier le résultat par l'élément	`résultat = 1`	`résultat = résultat * élément`
Fabriquer une liste avec les éléments	liste vide	appondre l'élément au résultat	`résultat = []`	`résultat.append(élément)`
Fabriquer une chaîne de caractères	chaîne vide	concaténer du texte à la fin de la chaîne	`résultat = ""`	`résultat = résultat + bout_de_texte`

Sommaire

1. **Utilité d’un algorithme, d’un programme**
   On se concentre sur le fait qu’un algorithme produit un résultat
2. **Programmes simples**
   Des programmes qui prennent des entrées et produisent un résultat immédiatement
3. **Accumulation**
   Produire un résultat petit à petit
4. **Valeur initiale et opération**
   Pour une opération répétée, on peut toujours trouver une valeur initiale qui correspond
5. **Exercices d’accumulation**
   Six exercices

Exercice 1 : somme des n premiers nombres

Exercice · 8 min

Écrire un programme qui :

- demande un nombre n ;
- calcule la somme des nombres de 1 jusqu'à n ;
- affiche cette somme.

Mettre en œuvre :

- vos connaissances sur l'itération ;
- vos connaissances sur l'accumulation, y compris la valeur initiale et l'opération répétée.

Réfléchir au nombre de variables dont vous allez avoir besoin.

Exercice 1 : somme des n premiers nombres

Écrire un programme qui demande un nombre `n`, calcule la somme des nombres de 1 jusqu'à `n` et affiche cette somme.

Il y a plusieurs manières de résoudre ce problème. En voici deux :

```python
n = int(input("donnez moi un nombre : "))

somme = 0
i = 1
while i <= n:
    somme = somme + i
    i = i + 1

print(somme)
```

```python
# On utilise l'idée que la somme
#  1, 2, 3, 4, 5, 6, ..... n est égale à
#  n, n-1, .............     3, 2, 1
#
# On calcule donc la somme en partant de n
# et en descendant jusqu'à 1

n = int(input("donnez moi un nombre : "))

somme = 0
while n >= 1:
    somme = somme + n
    n = n - 1

print(somme)
```

Le programme : somme-des-n-premiers-nombres-solution.py

Exercice 2 : calculer la *moyenne* d'une suite de nombres demandés

Exercice · 10 min

Le programme demande des nombres à la personne qui l'utilise, et s'arrête quand elle répond `0` (sans prendre en compte le 0 dans le calcul).

Un exemple de réponses successives serait : 4, 5, 4, 6, 0.
Dans ce cas, il faut que le programme calcule (4 + 5 + 4 + 6) / (1 + 1 + 1 + 1), c'est-à-dire 4.75.

À la fin, le programme affiche le résultat.

Le fichier qui calcule la somme de nombres demandés est en lien ci-dessous : commencer par le télécharger, puis le modifier pour qu'il calcule la moyenne.

Le fichier de base avec la somme de nombres demandés : somme-de-nombres-demandes.py

Conseils pour réaliser le programme :
Votre programme va devoir compter le nombre de nombres qui lui sont donnés, en plus de la somme des nombres. On accumule donc **deux choses en même temps**.

Exercice 2 : calculer la *moyenne* d'une suite de nombres demandés

Calculer la moyenne des nombres demandés. Pour les réponses 4, 5, 4, 6, 0 :

```
1  somme = 0
2  compte = 0
3  n = int(input("Entrez un nombre (0 pour terminer) "))
4  while n != 0:
5      somme = somme + n
6      compte = compte + 1
7      n = int(input("Entrez un nombre (0 pour terminer) "))
8  print("Moyenne :", somme / compte)
```

Le programme : [moyenne-de-nombres-demandes-solution.py]

ligne	somme	compte	n	affichage
1	0			
2		0		
3			4	
4				
5	4			
6		1		
7			5	
4				
5	9			
6		2		
7			4	
4				
5	13			
6		3		
7			6	
4				
5	19			
6		4		
7			0	
4				
8				Moyenne : 4.75

Exercice 3 : Batman le retour

Exercice · 6 min

Écrire un programme qui **fabrique la chaîne de caractères suivante :**

*na na na

Exercice 3 : Batman le retour

Écrire un programme qui fabrique la chaîne de caractères « na na na ... na ...Batman! », le `"na "` répété 100 fois.

```
s = ""

i = 1
while i <= 100:
    s = s + "na "
    i = i + 1

s = s + " ...Batman!"

print(s)
```

Le programme : batman-le-retour-solution.py

Exercice 4 : retourner une liste

Exercice · 8 min

Étant donné une liste, par *exemple* :

```
liste = [17, 89, 56, 23, 10]
```

Fabriquer une seconde liste, dans une variable `liste_inversée`, qui contient les éléments dans l'ordre inverse ; pour l'*exemple*, ça donne :

```
liste_inversée = [10, 23, 56, 89, 17]
```

À la fin, le programme affiche la liste, pour démontrer que l'algorithme est correct.

Conseils

Évidemment (et comme d'habitude), votre programme doit continuer de fonctionner même avec d'autres listes : il ne faut pas qu'il y ait de nombres fixés dans votre code.

Réaliser le programme *graduellement*. Si ça vous aide, vous pouvez commencer par écrire une toute petite partie du programme, sans boucle, éventuellement en faisant des copier-coller. Une fois que vous êtes sûr·es de votre approche, introduire la boucle.

Mettre en œuvre :

- vos connaissances sur l'itération des éléments d'une liste ;
- vos connaissances sur les valeurs initiales et les opérations pour les listes.

Exercice 4 : retourner une liste

Fabriquer une seconde liste, dans une variable `liste_inversée`, qui contient les éléments dans l'ordre inverse.

```python
liste = [17, 89, 56, 23, 10]

liste_inversée = []

while liste:
    e = liste.pop()
    liste_inversée.append(e)
    # pour bien voir comment l'algorithme progresse
    print("élément :", e, ", liste restante :", liste, ", liste_inversée :", liste_inversée)

print()
print(liste_inversée)
```

Le programme : retourner-une-liste-solution.py

Le programme affiche :

```
élément : 10 , liste restante : [17, 89, 56, 23] , liste_inversée : [10]
élément : 23 , liste restante : [17, 89, 56] , liste_inversée : [10, 23]
élément : 56 , liste restante : [17, 89] , liste_inversée : [10, 23, 56]
élément : 89 , liste restante : [17] , liste_inversée : [10, 23, 56, 89]
élément : 17 , liste restante : [] , liste_inversée : [10, 23, 56, 89, 17]

[10, 23, 56, 89, 17]
```

Exercice 5 : longueur de liste sans utiliser `len`

Exercice · 6 min

Étant donné une liste, par *exemple* :

```
liste = [17, 89, 56, 23, 10]
```

Écrire un programme qui détermine la longueur de la liste en comptant ses éléments.
Pour la liste donnée en *exemple*, c'est : 5

Pour cet exercice, **il est interdit d'utiliser la fonction `len`**.
Sinon, le programme ne mettrait pas en œuvre l'accumulation et se réduirait à :

```
print(len(liste))
```

Conseils

Évidemment (et comme d'habitude), votre programme doit continuer de fonctionner même avec d'autres listes : il ne faut pas qu'il y ait de nombres fixés dans votre code.

Réaliser le programme *graduellement*. Si ça vous aide, vous pouvez commencer par écrire une toute petite partie du programme, sans boucle, éventuellement en faisant des copier-coller. Une fois que vous êtes sûr·es de votre approche, introduire la boucle.

Mettre en œuvre :

- vos connaissances sur l'itération des éléments d'une liste ;
- vos connaissances sur le comptage en utilisant la technique de l'accumulation.

Exercice 5 : longueur de liste sans utiliser `len`

Écrire un programme qui détermine la longueur de la liste en comptant ses éléments, sans utiliser la fonction `len`.

```
liste = [17, 89, 56, 23, 10]

longueur = 0

while liste:
    longueur = longueur + 1
    liste.pop()
    print("longueur :", longueur, ", liste restante :", liste)

print()
print("Longueur de la liste :", longueur)
```

Le programme : [longueur-de-liste-sans-len-solution.py]

Le programme affiche :

```
longueur : 1 , liste restante : [17, 89, 56, 23]
longueur : 2 , liste restante : [17, 89, 56]
longueur : 3 , liste restante : [17, 89]
longueur : 4 , liste restante : [17]
longueur : 5 , liste restante : []

Longueur de la liste : 5
```

Exercice 6 : séparer une liste en deux

Exercice · 8 min

Étant donné une liste contenant des paires d'informations :

```
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]
```

Écrire un programme qui fabrique deux listes distinctes à partir de la liste donnée. À la fin, le programme affiche les deux listes qu'il a fabriquées :

```
Les mois : ['janvier', 'février', 'mars', 'avril', 'mai', 'juin', 'juillet', 'août',
'septembre', 'octobre', 'novembre', 'décembre']
Les jours : [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
```

Le fichier de base avec la liste : separer-une-liste-en-deux.py

Conseils

Évidemment (et comme d'habitude), votre programme doit continuer de fonctionner même avec d'autres listes : il ne faut pas qu'il y ait de nombres fixés dans votre code.

Fabriquer les deux listes petit à petit (elles commencent leur vie vides).

Itérer une seule fois la grande liste `mois_et_jours`.

Exercice 6 : séparer une liste en deux

Écrire un programme qui fabrique deux listes distinctes à partir de la liste donnée. À la fin, le programme affiche les deux listes qu'il a fabriquées :

```
mois_et_jours = [
    "janvier", 31, "février", 28, "mars", 31, "avril", 30,
    "mai", 31, "juin", 30, "juillet", 31, "août", 31,
    "septembre", 30, "octobre", 31, "novembre", 30, "décembre", 31,
]

liste_mois = []
liste_jours = []

while mois_et_jours:
    mois = mois_et_jours.pop(0)
    jours = mois_et_jours.pop(0)
    liste_mois.append(mois)
    liste_jours.append(jours)

print("Les mois :", liste_mois)
print("Les jours :", liste_jours)
```

Le programme : [separer-une-liste-en-deux-solution.py]

Sources

Images

Daderot – la Pascaline, la machine à calculer de Blaise Pascal, Mathematisch-Physikalischer Salon, Dresde (domaine public)
commons.wikimedia.org